DeltaFabric: Technical Specification

DeltaFabric is a decentralized, gossip-based synchronization layer for high-performance machine learning. This version implements a **Synchronous Relay Fabric** with loop prevention and **Batch Aggregation** to maximize memory throughput.

1. Optimizer State: The 2-Set Rule

DeltaFabric minimizes memory overhead by requiring only two versions of the model parameters. Unlike traditional decentralized methods that might store a copy of every neighbor's model, DeltaFabric only tracks what is currently being worked on and what has already been shared.

The Anchor-Active Duality

The core of the system relies on the interplay between two weight buffers. W_{active} is the "dirty" buffer, constantly moving due to local gradients. W_{anchor} is the "clean" reference point, representing the consensus state. By calculating the delta between them, we isolate exactly what new information this node has discovered.

State Variable	Managed By	Description
W_{active}	ML Framework	The "living" weights currently being optimized by local gradients and peer updates.
W_{anchor}	ML Framework	The "reference" weights representing the state of the model when it was last synchronized.
SyncInterval	DeltaFabric	The frequency (in steps) at which the local node generates its own unique updates.
α	DeltaFabric	The damping factor used to integrate peer knowledge into the local model.
K_{pct}	DeltaFabric	The fixed percentage of weights (the most significant ones) to be shared per sync event.
ϵ_{relay}	DeltaFabric	A numerical floor to stop the propagation of updates that have become too small to matter.
SeenTable	DeltaFabric	A local de-duplication ledger that tracks the latest sequence number seen from every node in the cluster.

2. The Communication Packet (Keyed Delta)

To allow nodes to act as "repeaters" without causing infinite data loops, every piece of information is keyed by its original creator. This allows the fabric to distinguish between "new information" and "echoes" of its own data returning through the mesh.

Deltas vs. Noise

In a mesh network, a packet might take multiple paths to reach the same destination. Without **Keyed Deltas**, a node would receive its own update back from a neighbor and think it was "new knowledge," leading to a feedback loop that causes weight explosion. By attaching an `Originator_ID` and a `Sequence_ID`, we treat every update like a unique "delta broadcast" that can be verified for freshness.

```
use rkyv::{Archive, Deserialize, Serialize};
use std::collections::HashMap;

#[derive(Archive, Deserialize, Serialize, Debug, PartialEq)]
#[rkyv(compare(PartialEq), derive(Debug))]
pub struct FabricPacket {
    /// Maps Originator_ID -> SparseDelta
    /// This allows a single network packet to contain updates from multiple source
    pub updates: HashMap<u64, SparseDelta>,
}

#[derive(Archive, Deserialize, Serialize, Debug, PartialEq)]
#[rkyv(compare(PartialEq), derive(Debug))]
pub struct SparseDelta {
    pub sequence_id: u64,
    pub indices: Vec<u32>,
    pub values: Vec<f32>,
}
```

3. Core Operations

A. Phase: Integration & Relay (Continuous with Batch Aggregation)

This phase occurs every time the fabric checks the network. It handles the "Gossip" part of the protocol. When a node receives a packet, it doesn't just apply it; it checks if the delta is fresh. If the information has a higher sequence ID than what is stored in the `SeenTable`, it is processed.

The Batch Aggregation Strategy

Decentralized ML often suffers from a "High Fan-In" problem: if a node has 100 neighbors, it might receive 100 updates per second. Updating a 1GB model 100 times would destroy training performance due to memory bus saturation. DeltaFabric solves this by using a temporary `aggregator`. We sum all peer changes into this small hashmap first, then apply the total result to the large weight buffers in a single, linear pass.

The Dual Update Rule

When peer deltas are accepted, they are applied to both W_{active} and W_{anchor}. This is a critical mathematical safeguard. By updating the anchor, the node effectively says: *"I have acknowledged this peer's progress; it is now part of my reference state."* If we only updated the active weights, the node would see the peer's update as a "change" it made itself and try to re-broadcast that peer's data as its own progress.

```
/// Processes an incoming packet and prepares a dampened relay.
///
/// Returns a map of deltas to relay and populates the `aggregator` with summed upd
pub fn process_and_collect_relay(
    aggregator: &mut HashMap<u32, f32>,
    incoming: &ArchivedFabricPacket,
    seen_table: &mut HashMap<u64, u64>,
    alpha: f32,
    relay_threshold: f32,
) -> Option<HashMap<u64, SparseDelta>> {
    let mut relay_updates = HashMap::new();

    for (&origin_id, delta) in incoming.updates.iter() {
        let last_seq = seen_table.get(&origin_id).unwrap_or(&0);

        // Loop Prevention: Only process "Fresh Deltas"
        if delta.sequence_id > *last_seq {
            seen_table.insert(origin_id, delta.sequence_id);

            let mut relay_indices = Vec::new();
            let mut relay_values = Vec::new();

            for (i, &idx) in delta.indices.iter().enumerate() {
                let val = delta.values[i];
                let damped_val = val * alpha;

                // Accumulate updates in memory-efficient map first
                *aggregator.entry(idx).or_insert(0.0) += damped_val;

                // Prepare relay data: Pruning Gate
                if damped_val.abs() >= relay_threshold {
                    relay_indices.push(idx);
                    relay_values.push(damped_val);
                }
            }

            if !relay_indices.is_empty() {
                relay_updates.insert(origin_id, SparseDelta {
                    sequence_id: delta.sequence_id,
                    indices: relay_indices,
                    values: relay_values,
                });
            }
        }
    }
```

```
    if relay_updates.is_empty() { None } else { Some(relay_updates) }
}
```

B. Phase: Local Generation (Top-K Selection)

Every $\mathrm{SyncInterval}$ steps, the node looks at the progress it has made locally.

Why Top-K instead of Magnitude Thresholds?

In early training, gradients are large. In late training (fine-tuning), gradients are tiny. If we used a fixed magnitude threshold (e.g., "only send changes > 0.1"), we would send everything at the start and nothing at the end. **Top-K Selection** ensures that we always send the top $K\%$ of indices that changed the *most* relative to other indices. This provides a consistent "information budget" per step, allowing the network to adapt to the scale of the gradients automatically.

```
/// Generates local delta by selecting the Top-K largest changes.
pub fn generate_local_delta(
    active: &[f32],
    anchor: &mut [f32],
    top_k_pct: f32,
    my_id: u64,
    seq: u64,
) -> Option<SparseDelta> {
    let mut deltas: Vec<(u32, f32)> = active.iter()
        .zip(anchor.iter())
        .enumerate()
        .map(|(i, (act, anc))| (i as u32, act - anc))
        .collect();

    // Sort by absolute magnitude descending to find most important updates
    deltas.sort_unstable_by(|a, b| b.1.abs().partial_cmp(&a.1.abs()).unwrap());

    let k = (active.len() as f32 * top_k_pct).ceil() as usize;
    let top_k = &deltas[..k.min(deltas.len())];

    let mut indices = Vec::new();
    let mut values = Vec::new();

    for &(idx, val) in top_k {
        if val != 0.0 {
            indices.push(idx);
            values.push(val);
            // Anchor Update: "Acknowledge" the sent delta so we don't re-send it n
            anchor[idx as usize] += val;
        }
    }

    if indices.is_empty() { None } else {
        Some(SparseDelta { sequence_id: seq, indices, values })
    }
```

```
}
```

4. The Fabric Session (Zenoh Abstractor)

The `Session` acts as the hardware-abstraction layer. It handles the low-level P2P discovery and network packet management, allowing the high-level `Fabric` logic to remain agnostic of the underlying transport protocol. It uses Zenoh's high-performance Pub/Sub to manage cluster states and weight deltas.

```
pub mod delta_fabric {
    use std::collections::{HashMap, HashSet, VecDeque};
    use std::time::Duration;
    use tokio::time::sleep;
    use serde::{Serialize, Deserialize};
    use zenoh::{Session as ZenohSession, Subscriber, publication::Publisher};

    #[derive(Serialize, Deserialize, Clone)]
    struct NodeState {
        expected_peers: Vec<u64>,
        status: String,
    }

    #[derive(Clone, Debug)]
    pub struct Node {
        pub id: u64,
        pub expected_peers: Vec<u64>,
    }

    pub struct Session {
        zenoh: ZenohSession,
        pub node: Node,
        delta_subscribers: HashMap<u64, Subscriber<'static, ()>>,
        pub delta_publisher: Option<Publisher<'static>>,
    }

    impl Session {
        pub async fn new(node: Node) -> Self {
            let zenoh = zenoh::open(zenoh::Config::default()).await.unwrap();
            Self { zenoh, node, delta_subscribers: HashMap::new(), delta_publisher:
        }

        /// Blocks until every node in the expected graph has announced "READY".
        pub async fn join_cluster(&mut self) {
            // ... Barrier logic implementation ...
        }

        pub fn pull_packets(&self) -> Vec<zenoh::sample::Sample> {
            let mut samples = Vec::new();
            for sub in self.delta_subscribers.values() {
                while let Ok(sample) = sub.try_recv() {
                    samples.push(sample);
```

```
                }
            }
            samples
        }

        pub async fn broadcast(&self, packet: crate::FabricPacket) {
            // ... Serialization and broadcast logic ...
        }

        pub async fn shutdown(&mut self) {
            // ... Departure logic ...
        }
    }
}
```

5. Burn Framework Integration (Optimized Loop)

The `Fabric` component is the primary interface for the AI developer. It orchestrates the entire synchronization workflow. It ensures that the heavy lifting of networking and sparsification happens with zero overhead to the training loop.

```
pub mod delta_fabric {
    pub mod burn {
        use super::Session;
        use std::collections::HashMap;
        use crate::{process_and_collect_relay, generate_local_delta, FabricPacket,
        use ::burn::module::Module;
        use ::burn::tensor::backend::Backend;

        pub struct Fabric {
            pub session: Session,
            pub anchor_weights: Vec<f32>,
            pub seen_table: HashMap<u64, u64>,
            pub top_k_pct: f32,
            pub threshold_relay: f32,
            pub sync_interval: u64,
            pub local_sequence: u64,
            pub alpha: f32,
        }

        impl Fabric {
            /// Synchronizes a Burn model with the DeltaFabric cluster.
            pub async fn step<B: Backend>(&mut self, mut model: Model<B>, step_coun
                let mut active_flat = Self::flatten_burn_model(&model);
                if self.anchor_weights.is_empty() {
                    self.anchor_weights = active_flat.clone();
                }

                let mut aggregator: HashMap<u32, f32> = HashMap::new();
                let mut relay_packet = FabricPacket { updates: HashMap::new() };

                // 1. COLLECT & AGGREGATE PEER DELTAS
```

```
                for sample in self.session.pull_packets() {
                    let payload = sample.payload().to_bytes();
                    if let Ok(incoming) = rkyv::access::<ArchivedFabricPacket, rkyv
                        if let Some(updates) = process_and_collect_relay(
                            &mut aggregator,
                            &incoming,
                            &mut self.seen_table,
                            self.alpha,
                            self.threshold_relay,
                        ) {
                            relay_packet.updates.extend(updates);
                        }
                    }
                }

                // 2. BATCH APPLY UPDATES
                for (idx, total_delta) in aggregator {
                    active_flat[idx as usize] += total_delta;
                    self.anchor_weights[idx as usize] += total_delta;
                }

                // 3. LOCAL TOP-K GENERATION
                if step_count % self.sync_interval == 0 {
                    self.local_sequence += 1;
                    if let Some(delta) = generate_local_delta(
                        &active_flat,
                        &mut self.anchor_weights,
                        self.top_k_pct,
                        self.session.node.id,
                        self.local_sequence,
                    ) {
                        relay_packet.updates.insert(self.session.node.id, delta);
                    }
                }

                // 4. BROADCAST
                if !relay_packet.updates.is_empty() {
                    self.session.broadcast(relay_packet).await;
                }

                Self::unflatten_burn_model(&mut model, &active_flat);
                model
            }

            fn flatten_burn_model<B: Backend>(model: &Model<B>) -> Vec<f32> {
                vec![] // Placeholder: Framework-specific tensor extraction
            }

            fn unflatten_burn_model<B: Backend>(model: &mut Model<B>, flat_data: &[
                // Placeholder: Framework-specific tensor loading
            }
        }
    }
}
```

6. Appendix: 3rd Party Crate Usage Reference

A. rkyv (Zero-Copy Serialization)

DeltaFabric uses **rkyv** because standard serialization is too slow for ML. rkyv allows us to map bytes directly into memory without "unpacking" them.

```
use rkyv::{deserialize, rancor::Error, Archive, Deserialize, Serialize};

#[derive(Archive, Deserialize, Serialize, Debug, PartialEq)]
#[rkyv(compare(PartialEq), derive(Debug))]
struct Test {
    int: u8,
    string: String,
    option: Option<Vec<i32>>,
}

fn main() {
    let value = Test {
        int: 42,
        string: "hello world".to_string(),
        option: Some(vec![1, 2, 3, 4]),
    };

    use rkyv::{api::high::to_bytes_with_alloc, ser::allocator::Arena};
    let mut arena = Arena::new();
    let bytes = to_bytes_with_alloc::<_, Error>(&value, arena.acquire()).unwrap();

    let archived = rkyv::access::<ArchivedTest, Error>(&bytes[..]).unwrap();
    assert_eq!(archived, &value);
}
```

B. Zenoh (P2P Mesh Networking)

Zenoh provides the gossip transport. It is significantly lighter than MQTT, making it ideal for the low-latency requirements of P2P training.

Subscriber Example

```
#[tokio::main]
async fn main() {
    let config = zenoh::Config::default();
    let session = zenoh::open(config).await.unwrap();
    let subscriber = session.declare_subscriber("node/*/delta").await.unwrap();

    while let Ok(sample) = subscriber.recv_async().await {
        let payload = sample.payload().try_to_string().unwrap_or_else(|e| e.to_stri
        println!(">> Received {} from '{}'", payload, sample.key_expr().as_str());
    }
}
```

C. Burn (Deep Learning Framework)

Example of a standard model architecture and training step implementation.

```
use burn::{
    nn::{
        BatchNorm, PaddingConfig2d,
        loss::CrossEntropyLossConfig,
        pool::{MaxPool2d, MaxPool2dConfig},
    },
    prelude::*,
    tensor::backend::AutodiffBackend,
    train::{ClassificationOutput, InferenceStep, TrainOutput, TrainStep},
};

#[derive(Module, Debug)]
pub struct Model<B: Backend> {
    conv1: ConvBlock<B>,
    conv2: ConvBlock<B>,
    dropout: nn::Dropout,
    fc1: nn::Linear<B>,
    fc2: nn::Linear<B>,
    fc3: nn::Linear<B>,
    activation: nn::Gelu,
}

impl<B: AutodiffBackend> TrainStep for Model<B> {
    type Input = MnistBatch<B>;
    type Output = ClassificationOutput<B>;

    fn step(&self, item: MnistBatch<B>) -> TrainOutput<ClassificationOutput<B>> {
        let item = self.forward_classification(item);
        TrainOutput::new(self, item.loss.backward(), item)
    }
}
```